MANCHESTER 1824
The University of Manchester

A Modular Real-Time Haptic Rendering System Integrating Physics-Based Simulation and OpenGL Visualisation

A dissertation submitted to The University of Manchester for the degree of

Bachelor of Engineering in Mechatronic Engineering

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

11349510

School of Engineering

Abstract

This dissertation presents the design, implementation, and evaluation of a low-cost real-time haptic rendering system integrating a two-degree-of-freedom physical interface with a custom simulation and visualisation framework. The work addresses the challenge of achieving stable bidirectional haptic interaction under practical constraints, including limited actuator capability and non-negligible communication latency.

The system combines embedded torque-controlled actuation, high-rate host–device communication, physics-based simulation, and proxy-based haptic rendering within a multi-threaded software architecture. Particular emphasis is placed on the integration of these components into a complete closed-loop interaction pipeline, rather than the performance of individual subsystems in isolation.

Experimental evaluation demonstrates that the system achieves update rates of approximately 1 kHz with low host-side processing latency (sub-millisecond), but is primarily constrained by a consistent communication round-trip delay of approximately 15–16 ms. Despite this, stable and controllable haptic interaction is achieved through a combination of architectural and control strategies, including snapshot-based communication, virtual coupling with damping, and actuator-side torque rate limiting.

The results show that communication latency, rather than computational performance, is the dominant factor limiting achievable haptic fidelity, imposing a practical bound on renderable virtual stiffness. Hardware limitations, including the absence of current sensing and transmission non-idealities, further constrain force accuracy and consistency.

Overall, the work demonstrates that stable real-time haptic interaction can be achieved on a low-cost platform, while highlighting the central role of communication architecture and control design in determining system performance. The findings provide insight into the trade-offs between cost, stability, responsiveness, and fidelity in practical haptic systems, and identify clear directions for future improvement.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on Presentation of Theses.

Acknowledgments

Use of AI Tools

Large language models (including ChatGPT) were used during this project as a supplementary tool to assist with language refinement, report structuring, and code debugging.

These tools were used to improve clarity, grammar, and organisation of written content, and to support troubleshooting during software development.

All technical content, system design, implementation decisions, analysis, and conclusions are my own work. All material included in this report has been reviewed and verified by me.

1 Introduction

1.1 Background and Motivation

Robotic manipulators are increasingly used in domains such as industrial automation, teleoperation, and robot-assisted surgery, where safe and effective interaction with physical environments is essential. Although visual feedback provides important global information about the workspace, it alone may be insufficient for developing fine motor control, accurate force regulation, and intuitive understanding of contact conditions.

Haptic interfaces address this limitation by enabling bidirectional physical interaction between a user and a virtual or remote environment. By rendering forces, such systems can communicate contact, constraint, and interaction information that is difficult to convey visually alone [1]. In robotic manipulation tasks, this is particularly valuable for training applications, where users must learn not only where to move, but also how to regulate force and respond to contact conditions. Prior work has shown that haptic feedback can improve task performance and reduce excessive applied force, particularly for less experienced users [2].

Despite these benefits, access to haptic systems remains limited in research and educational settings. Commercial interfaces provide mature and high-performance force feedback, but are typically priced substantially above academic budgets and are often distributed through vendor-specific SDK integration pathways [1]. This can restrict accessibility and make it difficult for researchers and students to experiment with alternative control strategies, custom simulation environments, or modified hardware configurations. As a result, there is strong practical motivation for low-cost and modular haptic platforms that are suitable for prototyping, experimentation, and robotics training research.

At the same time, achieving stable and convincing haptic interaction is technically challenging. Real-time haptic rendering commonly requires update rates on the order of 1 kHz together with low latency and carefully designed control structure in order to render contact stably and responsively [3]. The stability of haptic interaction is particularly sensitive to delay in the feedback loop, since communication and computation delays introduce phase lag and reduce the maximum achievable virtual stiffness [4]. Consequently, the design of a practical haptic system is not simply a matter of adding actuators to an input device: it requires coordinated consideration of mechanical design, embedded control, communication architecture, simulation coupling, and force rendering strategy.

1.2 Problem Definition

This dissertation addresses the design and evaluation of a *low-cost*, *real-time haptic interface system* suitable for robotic manipulation training and experimentation, with explicit consideration of the practical constraints that arise in implementation.

The specific challenge is not merely to construct a haptic input device, but to integrate multiple subsystems into a stable bidirectional interaction loop: user motion must be sensed at the device, transmitted to a host system, interpreted within a simulated environment, converted into interaction forces, mapped back into actuator commands, and rendered to the user in real time. In practice, each stage of this loop introduces constraints. Limited actuator capability affects the achievable force output, communication delay limits responsiveness and stable virtual stiffness, and software architecture determines whether the system remains extensible and maintainable.

Existing work demonstrates the value of haptic feedback and the feasibility of low-cost haptic hardware, but there remains a practical gap between simple hardware demonstrations and fully integrated systems that combine low-cost actuation, real-time communication, physics-based simulation, and explicit treatment of latency and stability. Educational and open-source devices have shown that affordable haptic hardware is possible [5], [6], and low-cost 2-DOF academic systems have also been demonstrated [7]. However, comparatively less emphasis is placed on complete end-to-end architectures in which hardware, embedded control, host-side simulation, rendering, and communication are developed together and evaluated as an integrated real-time system.

The problem addressed in this project is therefore defined as follows: *to design, implement, and evaluate a low-cost haptic rendering system that is sufficiently modular for experimentation, sufficiently responsive for real-time interaction, and sufficiently stable to demonstrate practical bidirectional haptic coupling with a simulated robotic environment*.

1.3 Aim and Scope of This Dissertation

The project focuses on demonstrating the feasibility of a complete end-to-end haptic architecture, covering: device design and construction; embedded firmware development for sensing, communication, and actuator control; implementation of a host-side framework integrating physics simulation, haptic rendering, and visualisation; and experimental evaluation of communication performance, timing, stability, and qualitative interaction behaviour.

The scope is intentionally limited. The dissertation does not attempt fully calibrated force output, commercial-level rendering fidelity, or a formal user study. Final hardware validation was performed with force feedback on one joint only, due to actuator failure during development. The principal contribution therefore lies in *system-level design, integration, and evaluation*, together with analysis of the practical constraints that limit achievable performance. As the results demonstrate, the dominant limitation is communication latency rather than host-side computation, and addressing this is identified as the primary route to higher-fidelity interaction.

1.4 Objectives

To achieve this aim, the project pursued six objectives: (i) design and construct a 2-DOF haptic device using low-cost brushless actuators and magnetic encoders; (ii) implement embedded firmware for encoder acquisition, actuator control, and bidirectional packet-based communication; (iii) develop a modular host-side software architecture supporting world management, physics simulation, haptic rendering, and OpenGL visualisation; (iv) implement proxy-based contact rendering and Jacobian-based force-to-torque mapping; (v) evaluate communication timing, safety behaviour, and qualitative haptic interaction performance; and (vi) assess the practical limitations of the architecture and identify directions for future improvement.

1.5 Contributions of This Work

Prior work on low-cost haptic platforms focuses primarily on hardware design [5], [6], [7] or on isolated software demonstrations, with comparatively little emphasis on fully integrated real-time systems in which hardware, firmware, communication, simulation, and rendering are co-developed and evaluated together. This work addresses that gap. Its principal contributions are:

- **A complete end-to-end haptic rendering system.** Custom hardware, embedded firmware, a packet-based communication protocol, and a modular host-side framework with real-time physics and OpenGL visualisation are integrated into a single bidirectional pipeline and evaluated as a unified system. This type of co-design and cross-layer evaluation is less commonly emphasised in existing low-cost academic haptic prototypes.
- **Demonstration of stable interaction under non-negligible communication delay.** Stable, controllable contact interaction is achieved despite a measured round-trip latency of approximately 15–16 ms — more than an order of magnitude above the 1 ms control timestep. Sta-

bility is maintained without formal passivity guarantees through the combined use of virtual coupling, damping, snapshot-based communication, multi-threaded decoupling, and an embedded torque slew-rate limiter.

- **Quantitative identification of transport delay, not computation, as the dominant performance constraint.** Instrumented runtime logging isolates host-side processing latency (mean 0.299 ms, 99th percentile 2.04 ms) from transport-layer delay, demonstrating that the communication interface, not the host processor, is the bottleneck. This finding directly motivates communication-architecture improvements as the primary route to higher haptic stiffness in this class of system.
- **Characterisation of USB CDC burst behaviour and a snapshot-based mitigation strategy.** Measured host-side packet inter-arrival times reveal strongly bimodal delivery: most packets arrive in near-simultaneous bursts separated by gaps of 14–16 ms, consistent with USB CDC buffering. A snapshot-based channel design is shown to decouple the haptic rendering loop from this irregular delivery pattern, preventing stale-state accumulation without requiring changes to the underlying communication stack.

2 Literature review

2.1 Haptic Interfaces for Robotics

As established in Section 1.1, haptic interfaces provide force feedback that complements visual information and supports fine motor control in robotic manipulation and training contexts. This section reviews the relevant literature on rendering techniques, stability analysis, and low-cost hardware platforms.

Commercial haptic devices such as the 3D Systems *Geomagic Touch* and *Touch X*, the *Force Dimension* product range, and the Haply *Inverse3 X* are widely used in research, training, and simulation contexts [1]. These systems provide kinesthetic force feedback and are typically distributed with proprietary SDK integration workflows, although some also expose lower-level APIs for custom software development.

However, they are generally sold at substantially higher price points than low-cost academic prototypes. For example, the Geomagic Touch and Touch X are publicly listed at approximately US$3,400 and US$13,400 respectively [8], [9], while other systems are commonly distributed via quotation-

based sales channels [10], [11]. In contrast, the system developed in this work has a total hardware cost of around £180, highlighting the potential for low-cost systems to provide accessible haptic functionality for research and training applications.

2.2 Haptic Rendering Techniques

Real-time haptic rendering has been extensively studied, with particular focus on contact-based interaction models. Two widely adopted approaches are virtual coupling and the virtual proxy (or "god-object") method introduced by Zilles and Salisbury and later extended by Ruspini *et al.* [3], [12].

In these approaches, the device state is coupled to a constrained proxy within the virtual environment, enabling stable rendering of contact and geometric constraints. Reliable performance typically requires high update rates on the order of 1 kHz, together with accurate tracking of device state and consistent mapping from Cartesian interaction forces to actuator torques [3]. In robotic implementations, these mappings are commonly derived using the manipulator Jacobian within operational-space control frameworks [13].

Haptic systems are commonly classified as impedance-type or admittance-type devices, depending on whether the device measures position and outputs force (impedance) or measures force and outputs motion (admittance). Most grounded haptic interfaces, including the system developed in this work, operate as impedance devices due to the relative ease of position sensing and actuation.

2.3 Stability in Haptic Systems

Stable haptic interaction imposes strict requirements on latency, control architecture, and numerical implementation. Adams and Hannaford demonstrated that virtual coupling can provide stability guarantees by decoupling device dynamics from the virtual environment under passive interaction assumptions [4].

More generally, stability in haptic systems is often analysed using passivity theory. Colgate and Hogan showed that discrete-time haptic interfaces can become unstable when energy is artificially generated within the feedback loop due to sampling, delay, or numerical approximation errors [14]. Their passivity framework establishes conditions under which a haptic system remains energetically passive, thereby ensuring stable interaction with arbitrary passive users and environments.

These results highlight the sensitivity of haptic systems to delay and discrete-time effects. In practical implementations, communication latency and asynchronous computation introduce additional energy into the system, reducing stability margins and limiting the achievable virtual stiffness. This is particularly relevant in systems where communication delay is non-negligible relative to the control timestep, motivating the use of control structures that explicitly manage dynamic coupling, damping, and energy injection in order to maintain stable interaction.

2.4 Low-Cost Haptic Hardware

Advances in low-cost components, including 3D-printed structures, microcontrollers, and magnetic encoders, have enabled the development of accessible haptic devices using widely available hardware. This has led to increasing interest in open and modular platforms for research and education [1].

The *Hapkit* platform developed by Martinez *et al.* demonstrates that functional haptic devices can be realised at very low cost (approximately US$50) using 3D-printed components [5]. Subsequent work extended this approach to modular multi-degree-of-freedom configurations, including both pantograph and serial mechanisms [6].

Similarly, Lavatelli *et al.* presented an open-source low-cost 2-DOF haptic device, demonstrating that multi-degree-of-freedom actuation can be achieved within an academic budget while maintaining useful interaction capability [7].

However, much of this work focuses primarily on hardware design or educational platforms. Comparatively less emphasis is placed on fully integrated systems that combine low-cost hardware with real-time communication, simulation coupling, and extensible software architectures.

2.5 Summary of Literature

The literature establishes that haptic feedback improves performance in robotic manipulation tasks and provides well-developed methods for stable force rendering, particularly through virtual coupling and passivity-based control frameworks. Low-cost hardware platforms have also demonstrated that accessible haptic devices are feasible.

However, there remains a gap between simplified hardware demonstrations and fully integrated

real-time systems that combine low-cost actuation, real-time communication, simulation, and extensible software design. Addressing this gap motivates the development of a low-cost haptic system in which hardware, communication, simulation, and rendering are co-designed and evaluated as an integrated real-time pipeline.

3 System Design and Implementation

This section describes the system design and realised implementation, covering the physical device, embedded control and communication, and the host-side software stack used for rendering, simulation, and force feedback.

The system is designed to prioritise real-time stability, modularity, and reproducibility within the constraints of a low-cost prototype. Stable haptic interaction requires high update rates (approximately 1 kHz) and low end-to-end latency; accordingly, the architecture assumes high-frequency embedded torque control and asynchronous host-side computation. Each major subsystem runs on its own thread and exchanges data through explicit message channels, reflecting a trade-off between cost, achievable force fidelity, and system stability, with particular emphasis on maintaining stable interaction under non-negligible communication latency.

3.1 System Overview

The proposed system implements a bidirectional haptic interaction loop consisting of four principal layers: the haptic device hardware, embedded control firmware, host-side simulation and haptic rendering, and graphical visualisation. A conceptual overview of the signal and force flow is shown in Fig. 1.

User motion is measured at the device joints and transmitted to the host in real time. The host computes interaction forces based on the virtual environment state, maps these to joint torque commands, and returns them to the device actuators, forming a closed-loop interaction. Time-critical control tasks are executed on the embedded system at high frequency, while simulation, rendering, and visualisation are performed asynchronously on the host.

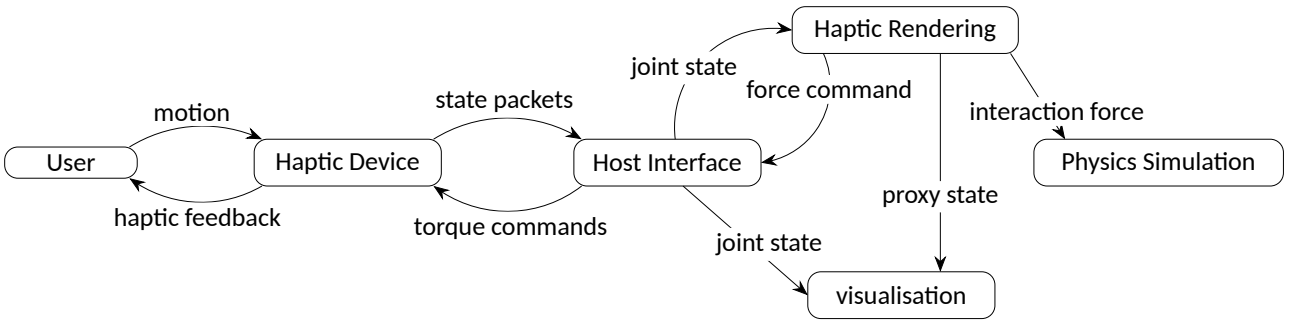


Fig. 1. High-level architecture of the proposed haptic system. User motion is measured by the haptic device and transmitted to the host interface. The host distributes state information to the haptic rendering and visualisation subsystems, while the haptic rendering module computes interaction forces, updates the simulation, and generates feedback commands that are returned to the device as actuator torque commands.

3.2 Mechanical and Embedded Platform

The physical interface is a planar two degree-of-freedom serial mechanism designed for low-cost fabrication and straightforward maintenance. A two-link layout was selected to provide a usable workspace while keeping actuator count, wiring complexity, and control overhead manageable.

The structure is primarily 3D printed, with each joint supported by steel shafts and rolling element bearings to reduce friction and improve backdrivability. Actuation is provided by brushless gimbal motors (T-Motor GB2208)[15]. The base joint incorporates a 3:1 GT2 belt reduction stage to increase available output torque, while the elbow joint is direct-drive to minimise mechanical complexity.

An adjustable idler pulley is integrated into the belt stage to enable tensioning and reduce backlash. Despite this, non-ideal behaviour such as compliance and occasional slip under high load was observed, as discussed in Section 6.3.

To ensure accurate joint angle measurement, absolute magnetic encoders (AS5048)[16] are mounted directly on the joint axes and interfaced via SPI. This is particularly important for the belt-driven joint, where transmission compliance would otherwise introduce discrepancies between motor-side and output angles.

For the belt-driven actuator, an additional AS5600 magnetic encoder[17] is mounted on the motor shaft and interfaced via I^2C. This provides rotor position feedback required for field-oriented control (FOC), while the joint encoder provides accurate output angle measurement.

A complete mechanical assembly, including component layout and link geometry, is provided in Ap-

pendix E.

Embedded control runs on an STM32 Nucleo-G431RB[18] platform with SimpleFOC Mini drivers and the SimpleFOC software stack [19]. Both actuators operate in torque-mode FOC [20]. A communication watchdog is implemented in firmware: if valid host commands are not received within the timeout window, output torques are forced to zero. Additionally, a torque slew-rate limiter is applied to each actuator command, constraining the rate of change of commanded torque to reduce high-frequency oscillations during rapid contact transitions.

Due to hardware constraints, no direct force sensing or current measurement was implemented; as a result, torque output is estimated rather than directly measured. Additionally, one actuator failed during development, meaning final hardware validation was conducted using single-axis actuation, although both joints remained available for sensing.

3.3 Communication Protocol and Device Interface

Host–device communication uses fixed-format binary packets over a USB virtual serial link at high update rate (target 1 kHz exchange). Two packet families are used: device state packets (firmware $\rightarrow$ host), containing sequence number, MCU timestamp, and joint angles; and torque command packets (host $\rightarrow$ firmware), containing command sequence, referenced state sequence, and commanded joint torques.

Sequence numbers detect dropped or out-of-order packets, while MCU timestamps provide a reference for timing analysis and latency measurement.

Both packet types include a two-byte header for stream re-synchronisation and a checksum for integrity checks.

On the host, serial I/O and protocol handling are split across two layers: *SerialLink* providing a low-level COM read/write abstraction, and *DeviceAdapter* handling packet parsing, forward kinematics, Cartesian force-to-joint torque mapping, and message-bus integration.

The device adapter runs in a dedicated 1 kHz loop. At each cycle, it retains the newest valid state packet, publishes tool pose and velocity to the haptic engine, consumes the latest wrench command, computes joint torques using the planar Jacobian transpose relation $\tau = J^T F$, and transmits the resulting torque command packet.

This ensures that control decisions are always based on the most recent available state, avoiding backlog-induced latency and improving responsiveness under high update rates.

3.4 Host Software Architecture

The host application is implemented as a modular multi-threaded C++ system. Major responsibilities are partitioned into the renderer, world manager, physics engine, haptic engine, and device adapter.

Inter-thread communication is implemented through a message bus with two channel types: queue channels for discrete commands and events (e.g., world edits, wrench messages), and snapshot channels for latest-value shared state (e.g., world snapshots).

This separation allows command traffic to be drained in batches while continuous state can be accessed with low latency.

3.4.1 World and Rendering Subsystems

WorldManager is the authoritative owner of virtual scene state. Objects store identity, geometry reference, pose, appearance, role labels, and physics properties. Subsystems do not mutate world data directly; instead, they publish commands which are applied centrally by the world manager.

Rendering is handled by an OpenGL subsystem running on the main thread. Each frame reads the latest world snapshot, resolves geometry and mesh handles, composes model transforms from object pose, and renders via shared shader and mesh abstractions. An ImGui interface provides runtime controls for object editing and parameter tuning; UI edits are emitted as world commands to preserve ownership boundaries.

3.4.2 Haptic Rendering Engine

The haptic engine executes at 1 kHz in a dedicated loop. At each cycle, it reads the latest world snapshot and newest tool state, performs contact queries, computes force output, and publishes results to device, physics, and visualisation channels.

Contact detection is performed using signed distance field (SDF) queries, with penetration depth and surface normals obtained from the local distance field gradient [21]. For each candidate ob-

ject, the tool position is transformed into object-local coordinates and queried against that object's SDF. If penetration is detected (negative signed distance), the tool point is projected to the surface to define the proxy position; otherwise, the proxy follows the tool in free space.

Force output is generated using a proxy-based virtual coupling model, in which a spring–damper relationship is defined between the device position and a contact-constrained proxy [4]:

$$\mathbf{F} = K\left(\mathbf{x}_p - \mathbf{x}_t\right) + D\left(\mathbf{v}_p - \mathbf{v}_t\right)$$

where $\mathbf{x}_p, \mathbf{x}_t$ are proxy and tool positions and $\mathbf{v}_p, \mathbf{v}_t$ are corresponding velocities. In the implemented configuration, $K = 500\,\mathrm{N\,m^{-1}}$, $M = 0.02\,\mathrm{kg}$, and $D = 0.7 \cdot 2\sqrt{KM}$.

This follows the standard virtual coupling formulation, although the present implementation does not explicitly enforce passivity-based design constraints. In particular, the proxy state is recomputed directly from the current tool position and contact geometry at each timestep, rather than being evolved as a constrained dynamic system. As a result, the implementation captures the force-generation behaviour of virtual coupling but does not realise the full passivity-based formulation described by Adams and Hannaford.

Accordingly, the force command is generated using a deliberately conservative coupling configuration, with damping and actuator-side limiting used to reduce abrupt force changes during contact transitions. The stability implications of this design are discussed in Section 6.2.

To maintain safe interaction, force magnitude is clamped to a maximum of 31 N before publication.

3.4.3 Physics Simulation Integration

Rigid-body dynamics are simulated using NVIDIA PhysX. The physics subsystem runs on its own simulation thread and consumes wrench messages from the haptic loop. Incoming forces are converted to impulses $\mathbf{J} = \mathbf{F}\,\Delta t$ over the effective timestep and applied at the contact point to the corresponding actor.

A fixed-step accumulator scheme is used to decouple simulation stability from variable external loop timing. When world topology or physics properties change, actors are rebuilt from the latest authoritative world snapshot. After each simulation update, dynamic poses are written back to the world manager so subsequent rendering and haptic queries operate on consistent state.

3.5 Logging and Communication Validation

To avoid introducing disk latency into the 1 kHz control path, runtime logging is performed asynchronously. The device thread publishes structured timing and state messages to dedicated logging channels, and a background logger thread drains these channels into memory.

At shutdown, logs are exported to two CSV files: `device_timing.csv`, containing matched state-command timing records for closed-loop control cycles including host-side timestamps associated with packet parsing, tool-state publication, wrench consumption, and torque command transmission; and `device_state_log.csv`, containing a record of every valid parsed incoming state packet including host receive timestamps, packet sequence identifiers, MCU timestamps, and measured joint angles.

A separate communication test executable (`serial_tests`) was also implemented to characterise serial performance independently of the full application. It supports two operating modes: a packet rate and integrity mode for analysing streamed device packets, and a ping–echo mode for measuring round-trip latency. In both modes, incoming serial data are processed using a buffered parser with explicit packet header search, checksum validation, and byte-wise re-synchronisation to ensure robustness to fragmented or corrupted input. Results are exported to machine-readable CSV files for offline analysis.

4 Experimental Methodology

This section defines the test procedures used to evaluate communication performance, closed-loop timing behaviour, simulation fidelity, safety behaviour, and hardware interaction characteristics of the proposed haptic rendering system.

The section focuses on what was tested and how measurements were collected; quantitative outcomes are reported in Section 5 and interpreted in Section 6.

4.1 Communication Performance Tests

Communication performance was characterised using the dedicated `serial_tests` utility described in Section 3.5. The tool supports two modes: a packet rate and integrity mode for analysing streamed

device packets, and a ping–echo mode for measuring round-trip latency. Both modes export CSV files for offline statistical analysis.

Tests were conducted across target transmission rates from 100 Hz to 2000 Hz to assess the effect of increasing packet frequency on reliability and timing performance.

4.1.1 Packet Integrity and Rate Tests

Packet integrity and communication timing were evaluated using the rate-test mode of the communication test utility.

For each target transmission frequency, the firmware was configured to stream fixed-format device state packets at a constant rate. Tests were performed at nominal frequencies of 100 Hz, 250 Hz, 500 Hz, 1000 Hz, 1250 Hz, and 2000 Hz.

For each test condition, the host continuously received and parsed incoming packets over the USB virtual COM interface. The recorded performance metrics included valid packet count, dropped packets inferred from sequence gaps, out-of-order packets, host-observed inter-arrival time, effective receive rate, and inter-arrival jitter.

4.1.2 Round-Trip Latency Test

Round-trip latency was evaluated using a ping–echo communication test.

In this test, the host transmitted timestamped ping packets to the embedded controller, which immediately returned an echo packet containing the original sequence number and timestamp.

Tests were performed using transmission intervals of 1 ms, 2 ms, and 10 ms. For each test, the evaluated metrics included mean, median, 95th percentile, and 99th percentile RTT, along with minimum and maximum RTT values and timeout counts.

These results were used to characterise communication latency and timing consistency under different transmission loads.

4.1.3 End-to-End Host-Side Timing Test

In addition to the standalone communication tests, end-to-end timing of the host-side haptic pipeline was evaluated using the runtime logging framework described in Section 3.5. Logs were captured during normal closed-loop operation and exported to `device_timing.csv` and `device_state_log.csv` for offline analysis in MATLAB.

The evaluation included host-side end-to-end latency between state-packet parsing and torque command transmission, intermediate pipeline delays between communication, haptic processing, and transmission stages, matched state-command sequence consistency, parsed state-packet inter-arrival timing, sequence gap statistics, and estimated parsed packet rate with MCU-side timing trends.

Because the host and microcontroller clocks were not synchronised, the analysis was restricted to host-side timing differences and trend-based interpretation of MCU timestamps. Absolute one-way latency between host and device could therefore not be inferred directly.

4.2 Simulation Validation

Simulation validation tests were designed to assess the correctness and qualitative behaviour of the haptic rendering model independently of physical actuator dynamics.

Two simulation-focused evaluations were performed. First, the virtual wall test assessed the unilateral contact response of the proxy-based virtual coupling model. Second, constrained motion along a virtual surface was evaluated to verify that the model generated forces predominantly normal to the surface while allowing tangential motion. Physical closed-loop interaction with actuator feedback was evaluated separately through the stable-contact and constrained-motion hardware tests in Sections 4.4.2 and 4.4.3.

4.2.1 Virtual Wall Test

A virtual wall test was performed to validate the basic unilateral contact behaviour of the haptic rendering engine. A planar contact boundary was defined in the virtual environment, and the device end-effector was manually moved toward and against the wall during repeated trials.

This test verified that the proxy-based rendering method prevented penetration of the constrained proxy through the wall surface while allowing the unconstrained device position to continue moving, thereby generating a restoring virtual coupling force.

The evaluation examined the absence of significant rendered force outside the wall, the increase in force with penetration depth, and qualitative agreement with the implemented spring–damper coupling model.

4.2.2 Constrained Motion Along a Virtual Surface

The system was further evaluated for its ability to support constrained motion along a virtual surface.

During this test, the end-effector was brought into contact with a planar wall and allowed to move along the surface while maintaining contact. This behaviour arises naturally from the proxy-based rendering method, where motion normal to the surface is resisted while tangential motion remains unconstrained.

The evaluation assessed the ability to maintain contact during motion, the absence of resistance to tangential movement, stability during sustained sliding interaction, and smooth transitions between free-space and constrained motion.

4.3 Safety Validation

Safety validation tests ensured that the system remained bounded and behaved predictably under both normal and fault conditions. These tests focused on verifying torque saturation, rate limiting, and watchdog-based fail-safe behaviour.

4.3.1 Torque Saturation Test

A torque saturation test verified that actuator command limits were correctly enforced across both host and embedded layers.

Force outputs were limited on the host side to a predefined maximum, and the embedded controller applied an additional clamp to ensure commands remained within safe bounds. Testing confirmed that host-side force commands were correctly limited, transmitted torque commands were

further bounded by the embedded controller, and actuator output remained within safe and predictable limits.

4.3.2 Rate Limiting Test

A rate limiting test evaluated the effect of constraining the rate of change of commanded torque at the embedded controller level.

A slew-rate limiter was implemented to reduce high-frequency oscillations and improve stability during rapid contact transitions. Testing verified that rapid command changes were smoothed into bounded torque ramps, high-frequency oscillations were reduced during contact events, and overall system stability improved during interaction.

4.3.3 Watchdog Timeout Test

A watchdog timeout test verified safe system behaviour under communication loss by interrupting the incoming command stream during operation.

The embedded controller response confirmed that the watchdog triggered reliably when command updates ceased, commanded torque was removed within the intended timeout window, and the device returned to a safe passive state.

4.4 Hardware Validation

Hardware validation assessed the behaviour of the physical actuation system and overall device interaction, focusing on qualitative evaluation of responsiveness and stability.

4.4.1 Actuator Torque Response Characterisation

Basic actuator response behaviour was evaluated using standalone motor testing prior to system integration. Step changes in commanded torque were applied, and the resulting mechanical response was assessed qualitatively.

The evaluation examined responsiveness to torque changes, presence of oscillatory behaviour, smoothness of motion, and repeatability across repeated inputs. This represents qualitative validation rather than precise quantitative characterisation.

4.4.2 Stable Contact Test

A stable contact test evaluated the behaviour of the full closed-loop system during sustained interaction with a virtual surface.

The end-effector was brought into contact with a virtual object and maintained in contact while observing the resulting force feedback behaviour. The evaluation assessed the ability to maintain continuous contact without instability, the presence of oscillation or chatter, consistency of the rendered resistive force, and overall usability of the interaction.

4.4.3 Constrained Motion Hardware Test

Hardware constrained-motion behaviour was evaluated during physical operation of the device.

The end-effector was brought into contact with a planar virtual surface and moved along it while maintaining contact. The evaluation assessed whether the system produced forces predominantly normal to the surface, resisted penetration, and allowed motion tangential to the constraint without instability.

In addition, the device was released near the virtual surface to assess whether the rendered contact force could support the end-effector passively in a gravity-like interaction scenario.

5 Results

This section reports the measured outcomes of the Section 4 tests. It presents communication timing, simulation behaviour, safety outcomes, and hardware interaction observations.

Interpretation of these findings and their design implications is deferred to Section 6.

5.1 Communication Performance

Communication performance was evaluated using standalone serial tests and runtime logging of the integrated haptic application.

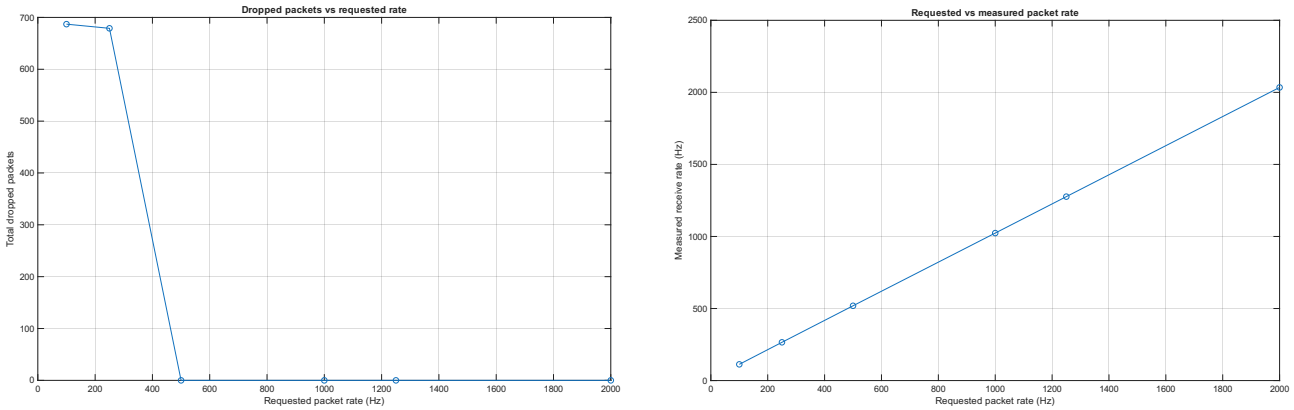
5.1.1 Packet Rate and Integrity Results

Packet rate testing showed that communication performance remained stable over a wide range of requested transmission frequencies, although the behaviour was not uniform across all tested rates.

At lower requested rates of 100 Hz and 250 Hz, packet loss was observed, with drop fractions of approximately 20.3% and 8.5% respectively. In contrast, for requested rates of 500 Hz and above, no dropped packets were observed in the recorded data, as shown in Fig. 2(a), where packet loss occurs only at the lowest transmission rates. Out-of-order packets were extremely rare across all higher-rate tests, with only a single out-of-order event recorded in each case.

Packet loss was more pronounced at lower transmission rates, a counterintuitive finding that may reflect the influence of host-side scheduling on USB buffer flushing behaviour; this is discussed further in Section 6.1.

The measured receive rate tracked the requested transmission rate closely across the full test range. As shown in Fig. 2(b), the relationship between requested and measured rate was approximately linear, indicating that the communication system maintained throughput proportional to the commanded transmission frequency.



(a) Dropped packets **(b)** Rate tracking

Fig. 2. Communication performance across transmission rates. (a) Total dropped packets as a function of requested rate, showing packet loss only at low frequencies. (b) Measured receive rate versus requested transmission rate, demonstrating approximately linear tracking.

Inter-arrival jitter, measured as the standard deviation of packet inter-arrival time, was present in all tests, with standard deviation decreasing as the requested rate increased, as shown in Fig. 3. However, percentile analysis revealed that occasional large timing excursions persisted even at

higher rates, indicating intermittent latency spikes. While average jitter decreased with increasing rate, worst-case timing excursions remained present.

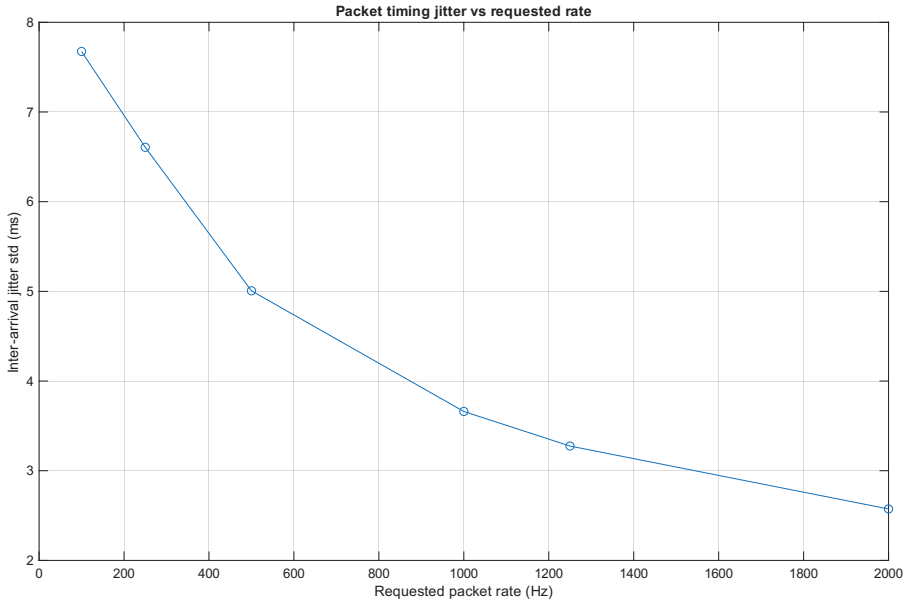


Fig. 3. Standard deviation of packet inter-arrival time as a function of requested transmission rate. Jitter decreases with increasing rate, although occasional large timing excursions remain present.

Overall, the serial link supported packet rates at or above 1 kHz with negligible packet loss in the tested configuration.

5.1.2 Round-Trip Latency Results

Round-trip latency results were highly consistent across all tested ping intervals. For nominal transmission intervals of 1 ms, 2 ms, and 10 ms, the success rate was 100% in all cases, with no timeouts observed over 1000 samples per test.

The mean measured round-trip latency remained approximately constant across all three test conditions, with values of 15.60 ms, 15.64 ms, and 15.67 ms respectively. Similarly, the 95th and 99th percentile RTT values were tightly grouped around 16.2–16.4 ms, indicating stable and predictable timing behaviour across repeated measurements. This behaviour is further illustrated in Fig. 4, where the cumulative distributions for all three tests overlap closely and rise sharply at approximately 15–16 ms, indicating that RTT is largely independent of the selected ping interval.

The standard deviation of RTT remained below 0.52 ms in all cases, suggesting relatively low variability in round-trip timing. Minimum and maximum observed RTT values showed occasional larger deviations from the typical latency range, particularly in the 1 ms test, but these did not significantly affect the overall distribution.

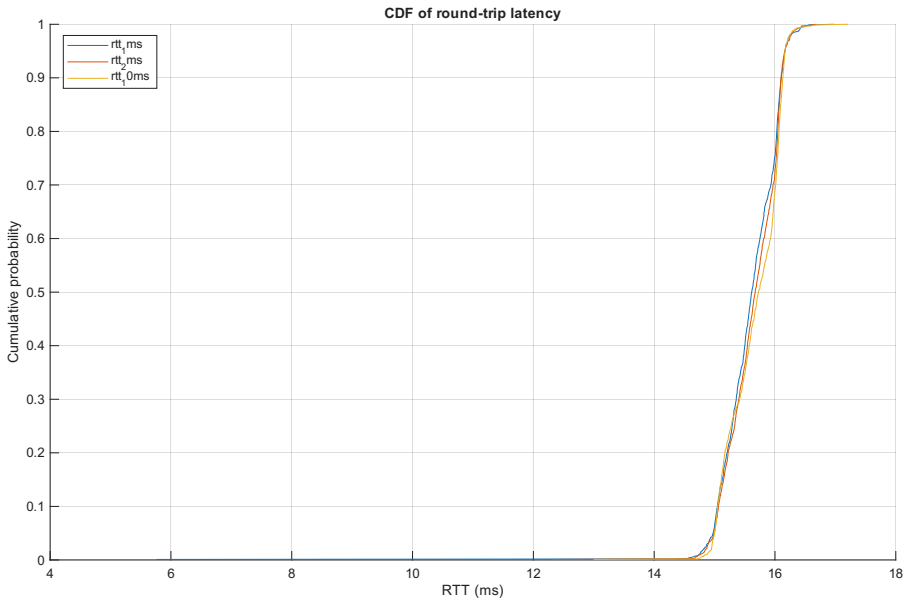


Fig. 4. Cumulative distribution functions of measured round-trip latency for nominal ping intervals of 1 ms, 2 ms, and 10 ms. The distributions overlap closely and rise sharply at approximately 15–16 ms, indicating that round-trip latency is dominated by a largely fixed delay component rather than by the selected ping interval.

These results indicate that RTT was dominated by a largely fixed delay component and changed little with ping interval. The measured delay remained predictable due to low variability and zero observed timeouts.

5.1.3 End-to-End Host-Side Timing Results

End-to-end timing behaviour of the integrated haptic pipeline was evaluated using runtime logging of matched state-command cycles and raw parsed state packets.

Analysis of `device_timing.csv` showed that all recorded control cycles were successfully matched, with a matched fraction of 1.000, indicating consistent correspondence between received state packets and transmitted torque commands.

The host-side end-to-end latency, defined as the time between state-packet parsing and completion of torque command transmission, was found to be low. The mean latency was 0.299 ms, with a median of 0.104 ms. The 95th percentile latency was 0.546 ms, and the 99th percentile was 2.04 ms, with a maximum observed latency of 3.35 ms. As shown in Fig. 5, the distribution was concentrated at low values, with prominent peaks near 0 ms and around 0.5 ms, and only a limited number of higher-latency events.

Breakdown of intermediate pipeline delays showed that the majority of this latency was associated

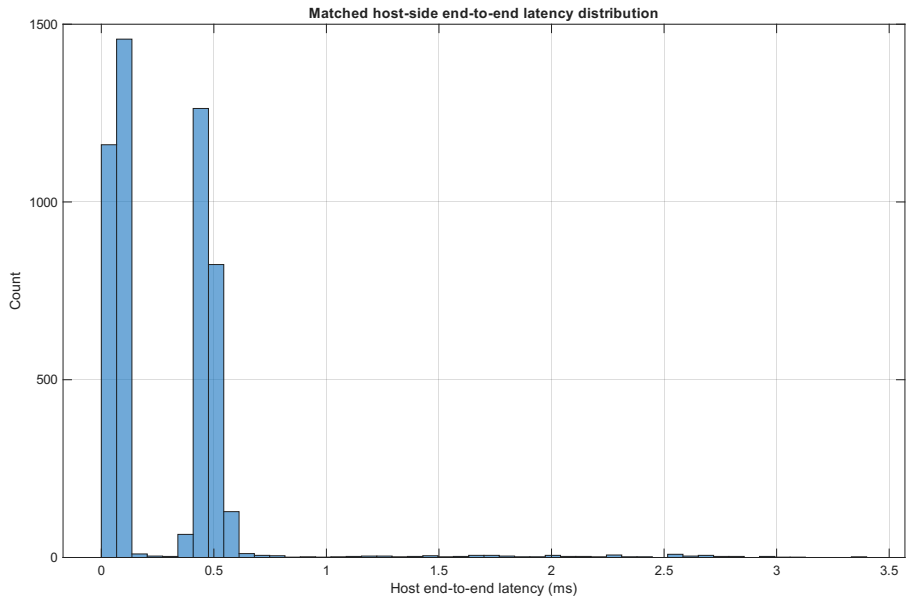


Fig. 5. Distribution of matched host-side end-to-end latency, measured from state-packet parsing to completion of torque command transmission. The latency is concentrated at low values, with prominent peaks near 0 ms and around 0.5 ms, and only a small number of higher-latency outliers.

with serial transmission. The mean transmission duration was 0.249 ms, while the delay associated with publishing tool state and processing the haptic pipeline remained small, with mean values of 0.024 ms and 0.0044 ms respectively. This indicates that host-side computation and message-passing overhead were negligible relative to communication time.

Sequence analysis showed that command sequence progression remained consistent, with a mean command sequence gap of 1.14 and a maximum of 6, indicating that command updates were generally applied sequentially with minimal skipping. In contrast, the matched state sequence gap was significantly larger, with a mean of 33.45. This reflects the fact that the control loop operated on a subset of the available incoming state packets rather than processing every parsed packet. Here, sequence gap denotes the difference between consecutive sequence identifiers, such that a value of 1 corresponds to consecutive packets or commands and larger values indicate skipped intermediate sequence numbers.

A total of 54 161 valid state packets were parsed during the test, compared to 5 048 matched control cycles, confirming that the control loop operated on a subset of the incoming packet stream rather than consuming every parsed packet. The estimated parsed packet rate, computed from the mean host-side inter-arrival time, was approximately 1006 Hz, which is close to the expected 1 kHz transmission rate.

Analysis of host-side parsed-packet inter-arrival timing revealed significant variability. The mean inter-arrival time was 0.994 ms, but the median was only 0.0016 ms, reflecting that many packets

were processed in immediate succession within bursts. The large difference between mean and median inter-arrival time indicates that packets were frequently processed in rapid bursts following periods of delay, rather than arriving at uniform intervals. This behaviour is illustrated in Fig. 6, where a dominant peak near 0 ms is accompanied by a much smaller secondary concentration around 14–16 ms. The 95th and 99th percentile inter-arrival times were 14.5 ms and 15.8 ms respectively, confirming the presence of intermittent host-side stalls.

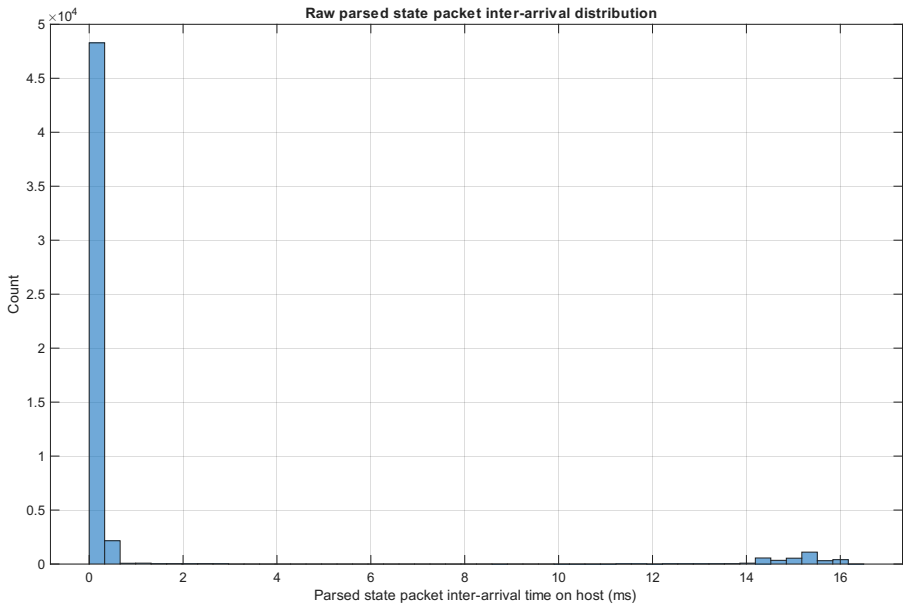


Fig. 6. Distribution of host-side inter-arrival time between consecutively parsed state packets. A dominant peak near 0 ms and a smaller concentration around 14–16 ms indicate burst-like packet processing at the host, consistent with buffered delivery and intermittent scheduling delays.

Despite this non-uniform host-side processing pattern, analysis of MCU-side timestamps showed that the embedded system maintained a stable and consistent update rate. The effective MCU period per state was approximately 1.00 ms, corresponding to an update rate of 1000 Hz, with very low variability.

Taken together, packet generation on the embedded controller was highly regular, while host-side packet processing was non-uniform and burst-like. Despite this, host-side processing latency remained low.

5.2 Simulation Validation Results

Simulation validation tests were conducted with actuator output disabled, allowing the behaviour of the haptic rendering model to be evaluated independently of hardware dynamics and communication effects.

5.2.1 Virtual Wall Model Response

A virtual wall test was performed to evaluate the output forces generated by the haptic rendering engine in isolation from physical feedback.

The rendered force exhibited the expected unilateral contact behaviour. No significant force was produced when the end-effector remained outside the virtual boundary, while penetration resulted in a restoring force that increased with penetration depth, consistent with the implemented spring–damper virtual coupling model. This behaviour matches the expected response of a unilateral contact model, in which force is applied only during penetration and acts along the surface normal.

However, this behaviour degraded under conditions of large penetration. In these cases, both the magnitude and direction of the rendered force became inconsistent. This is attributed to limitations of the projection-based contact model, where the closest-point computation becomes less reliable at greater distances from the surface. As a result, discontinuities in the estimated surface normal and projection point can arise, leading to reduced or misdirected restoring forces.

This behaviour is consistent with known limitations of simplified projection-based proxy methods, where the proxy is recomputed from the current tool position without enforcing continuous constraint satisfaction. The implications for interaction stability and force fidelity are discussed in Sections 6.2 and 6.4.

Overall, this test confirms that the contact model produces physically consistent behaviour under typical interaction conditions, while highlighting limitations under extreme penetration scenarios.

5.2.2 Constrained Motion Model Behaviour

The behaviour of constrained motion was also evaluated with actuator output disabled.

When the end-effector was moved along the virtual wall surface, the computed force remained predominantly normal to the surface, resisting motion into the wall while allowing tangential motion. This behaviour is characteristic of virtual fixture or constraint-based haptic rendering, where motion is restricted in selected directions while remaining unconstrained in others. Such behaviour is particularly relevant for training applications, where users must learn to follow constrained paths or avoid forbidden regions.

These results demonstrate that the simulation model can reliably generate constraint forces aligned with surface geometry, supporting its suitability for guidance and training-based interaction tasks.

5.3 Safety Validation Results

Safety mechanisms were evaluated primarily as functional pass/fail protections.

5.3.1 Torque Saturation

Torque saturation mechanisms were effective in maintaining bounded actuator behaviour. Under conditions where large forces were generated by the haptic rendering engine, the host-side force clamp limited the magnitude of commanded forces before transmission. This was further reinforced by an embedded torque clamp, ensuring that actuator outputs remained within predefined safety bounds.

5.3.2 Rate Limiting

The implemented slew-rate limiter on the microcontroller significantly improved stability during interaction. Rapid changes in commanded torque were smoothed into gradual transitions, reducing high-frequency oscillations and improving the perceived stability of contact.

Without rate limiting, abrupt force changes resulted in noticeable vibration and unstable behaviour. The inclusion of the rate limiter reduced these effects and contributed to stable sustained interaction.

Overall, force clamping and rate limiting maintained bounded and stable actuator behaviour in the tested conditions.

5.3.3 Watchdog Timeout Behaviour

The watchdog was triggered consistently when command updates were absent for longer than the configured timeout interval of 10 ms.

When the stream of incoming torque commands from the host was interrupted, the embedded controller detected the absence of updates and reset actuator torque commands to zero after the predefined timeout interval.

This resulted in the device returning to a passive, unpowered state without exhibiting unintended motion or sustained force output.

The watchdog behaviour was observed to be consistent and repeatable across multiple trials, providing a robust safeguard against communication failure or host-side faults.

These results confirm effective fail-safe behaviour during communication disruption.

5.4 Hardware Validation Results

Hardware validation focused on qualitative assessment of responsiveness, stability, and perceived interaction behaviour during representative haptic tasks. Because direct force measurement and current sensing were not available, the hardware results should be interpreted as evidence of qualitative interaction capability rather than calibrated force accuracy. In addition, final force-feedback testing was limited to single-axis actuation due to actuator failure, although both joints remained available for position sensing.

5.4.1 Actuator Torque Response Characterisation

Standalone testing of individual actuators demonstrated that the motor and control hardware were capable of producing responsive and stable torque output.

Step changes in commanded torque resulted in immediate and clearly observable changes in actuator behaviour. Although the actuator control did not provide a calibrated one-to-one relationship between commanded and physical torque, the response was generally smooth and repeatable, with limited overshoot and no sustained oscillatory instability under typical operating conditions.

Some minor oscillations were observed at higher torque levels or during rapid command changes; however, these were mitigated through the use of torque rate limiting and appropriate control parameter selection.

Overall, the qualitative assessment indicated that the actuator response was sufficiently fast and stable for use in haptic rendering applications, where timely and predictable force output is required.

5.4.2 Stable Contact Behaviour (Virtual Wall Interaction)

This test represents the physical realisation of the virtual wall interaction previously validated in Section 5.2.1.

The integrated system was able to support sustained contact interaction without loss of stability.

During testing, continuous contact with a virtual surface could be maintained for extended periods without runaway motion or divergence. The rendered force remained consistent, and the interaction was perceived as stable and controllable.

Some low-level vibration and noise were present during contact, particularly under high stiffness settings. However, this did not lead to instability and was reduced through the combined use of damping and torque rate limiting.

The system remained responsive to user input while maintaining a stable resistive force, indicating that the closed-loop architecture is suitable for practical haptic interaction scenarios.

These results demonstrate stable contact interaction under real-time conditions despite communication variability.

It is noted that the large penetration effects observed during simulation validation were less prominent during physical interaction. In practice, the rendered contact force limits penetration depth by opposing user motion. However, due to the finite torque output of the actuators and the achievable virtual stiffness, some penetration still occurs during contact.

This effect is more pronounced with small or thin virtual objects, where small physical displacements can correspond to significant virtual penetration.

Although stable contact behaviour was achieved, the rendered force did not exhibit perfectly rigid characteristics. The interaction was observed to have a degree of compliance, with force increasing gradually rather than instantaneously. This behaviour reflects the combined effect of the torque slew-rate limiter, actuator torque capability limitations, control fidelity, and transmission behaviour, which are discussed further in Sections 6.3 and 6.4.

Under attempts to push strongly through a rigidly rendered object, transmission non-idealities were also observed at the base joint. In particular, belt slip occurred under sufficiently high load, leading to a reduction in perceived resistance and a loss of consistency in the rendered force during ex-

treme contact conditions. Although this behaviour was not dominant during typical interaction, it became apparent during aggressive push-through attempts and further limited the maximum practically renderable stiffness.

5.4.3 Stable Contact Behaviour (Constrained Motion)

Constrained motion behaviour was also evaluated under physical operation of the device. Given that physical actuation was available only on the first joint during final testing, the constrained-motion behaviour observed in hardware represents a partial implementation of the intended two-axis guidance behaviour.

When the end-effector was brought into contact with a virtual surface and moved along it, the system generated forces predominantly normal to the surface, resisting penetration while allowing motion tangential to the constraint. This resulted in guidance-like behaviour consistent with the constrained motion model validated in simulation.

An additional observation was made when the device was released near a virtual surface. When positioned close to the surface, the end-effector could remain supported by the rendered contact force, behaving similarly to a physical object resting against a rigid boundary. In contrast, when released further from the surface, the end-effector moved freely until contact was re-established, at which point motion became constrained along the surface.

This behaviour demonstrates that the system is capable of producing physically intuitive constraint forces, while also reflecting the finite stiffness and force limits of the actuator system. The transition between free motion and constrained interaction was smooth and did not result in instability.

The guidance behaviour was effective but not perfectly rigid, with some compliance and variation in constraint force observed during motion. This was consistent with the limited actuator stiffness and partial actuation available in the final hardware configuration.

These results confirm stable and intuitive constraint interaction under real-time conditions.

5.5 Summary of Results

Overall, the system achieved stable real-time interaction on a low-cost platform.

The main measured constraints were fixed communication latency (approximately 15–16 ms), finite actuator capability, partial actuation in final testing, and transmission non-idealities under high load.

Within these constraints, the tests showed low packet loss at high update rates, low host-side processing latency, consistent safety behaviour, and stable qualitative contact/constraint interaction.

6 Discussion

6.1 Communication and Latency Constraints

The central finding from the communication evaluation is that transport delay — measured at approximately 15–16 ms round-trip — is the dominant limitation on closed-loop performance, not host-side computation. Host-side processing latency remained sub-millisecond in the vast majority of cycles, confirming that the computational architecture performs well within the requirements of 1 kHz haptic control.

The consequence is that achievable virtual stiffness is bounded by the communication path rather than by processing speed. Delayed force feedback introduces phase lag into the closed-loop system, reducing stability margins and constraining safe virtual stiffness without additional stabilisation mechanisms.

Delay management must therefore be treated as a first-order design constraint, and improving the communication architecture is the primary route to higher haptic fidelity in this class of system.

Nevertheless, stable haptic interaction remained achievable with appropriate architectural and control measures.

A likely contributing factor is the use of the ST-Link virtual COM interface for host–device communication. This interface supported rapid development, but the observed burst-like delivery pattern suggests buffering and scheduling overhead that is large relative to the control timestep.

While the present work does not isolate the exact contribution of each stage in the communication path, the evidence suggests transport-layer behaviour is a major source of delay. Lower-latency communication options could therefore provide a direct path to improved contact fidelity.

These results highlight that, in practical haptic systems, communication architecture can be a dominant performance constraint, and must be considered alongside control and mechanical design when targeting high-fidelity interaction.

6.2 Stability of the Haptic Interaction

Although communication timing was imperfect, the system maintained stable interaction across tested conditions. The key point is that stability resulted from architectural choices that reduced the control impact of transport irregularities rather than from a formally guaranteed passive interconnection.

Firstly, snapshot-based communication channels ensured that the haptic loop operated on the most recent available state rather than queued historical samples. This design choice limited the destabilising effect of burst delivery by preventing stale data from accumulating in the feedback path.

Secondly, the separation of communication, haptic rendering, and physics simulation into independent threads allows each subsystem to operate at an appropriate rate without blocking time-critical operations. This decoupling reduces the propagation of timing variability between subsystems.

In addition, damping and torque rate limiting at the embedded level were central to maintaining controllable contact. The virtual coupling introduced dissipation to counter delayed feedback effects, while the slew-rate limiter prevented abrupt torque transitions that can excite oscillations.

These stabilisation mechanisms can be interpreted within a passivity-based framework, in which communication delay and discrete-time effects introduce artificial energy into the feedback loop. The addition of damping and rate limiting acts to dissipate this energy, helping to preserve stable interaction despite the presence of significant communication latency. However, it is important to note that this behaviour reflects practical or empirical stability rather than a formally derived passivity condition.

Together, these mechanisms enable the system to maintain stable and controllable interaction despite communication delays that would otherwise limit performance in a purely high-stiffness haptic rendering framework.

6.3 Hardware Limitations

The hardware results must be interpreted within clear prototype constraints. These constraints do not invalidate the architectural findings, but they do bound the achievable force fidelity and the generalisability of quantitative conclusions.

A primary limitation was the absence of current sensing in the actuator control system. The initial design intended to utilise integrated motor driver solutions based on the STM32 B-G431-ESC1 platform, which provide built-in current sensing capabilities suitable for accurate torque control. However, significant integration challenges were encountered when attempting to interface these drivers with the selected magnetic encoders, particularly in achieving reliable and stable operation within the required control framework.

Due to the increased implementation complexity and project time constraints, a design decision was made to adopt SimpleFOC Mini driver modules, which provided a more straightforward and reliable integration pathway with the STM32 Nucleo development board. This allowed development effort to be focused on the embedded control and host-side software architecture, which formed a central objective of the project.

However, the use of SimpleFOC Mini drivers without current sensing introduces an important limitation: commanded torque cannot be interpreted as a calibrated physical torque output. Torque control is therefore effectively open-loop with respect to current, making output dependent on motor and load conditions. Consequently, the hardware evaluation should be read primarily as evidence of qualitative interaction capability rather than absolute force accuracy.

A second limitation arises from the mechanical transmission used to increase actuator torque. A GT2 timing belt reduction was implemented on the base joint to compensate for the limited torque output of the selected low-cost gimbal motors. While effective in increasing available torque, this transmission introduced non-ideal behaviour under high load conditions. During interaction with large rendered forces, the belt was observed to slip when the applied load exceeded the transmission capability. An adjustable idler pulley was introduced to increase belt tension and reduce slip; however, this did not fully eliminate the issue. Residual slip is likely due to the use of 3D-printed pulleys, which do not achieve the same tooth profile precision and engagement as manufactured components.

In addition, the overall torque capability of the selected motors was insufficient for convincingly

rigid contact rendering. Even without transmission slip, actuator limits imposed compliance, constraining the maximum practically renderable stiffness.

A further limitation arose from the failure of one actuator during development. As a result, final hardware validation used single-joint actuation with two-joint sensing. This reduced force-feedback dimensionality during physical testing, so conclusions about full 2-DOF force rendering remain provisional.

Overall, these results demonstrate that, in addition to communication latency, actuator capability and transmission design impose significant constraints on achievable haptic performance in low-cost systems.

6.4 Design Trade-Offs

The design of the system reflects a set of trade-offs between stability, responsiveness, and achievable force fidelity. While damping and torque rate limiting were effective in ensuring stable interaction, they reduce perceived stiffness and responsiveness. In particular, rate limiting of actuator torque results in a finite rise time when contact forces are applied, leading to a short delay between initial contact and the development of the full resistive force. This reduces the sharpness of hard-contact interactions such as collisions with rigid virtual walls, as observed during the hardware validation in Section 5.4.2.

As a result, high-speed interactions and abrupt contacts are not rendered with the same fidelity as would be expected in systems with lower latency and higher achievable stiffness. Instead, contact forces are perceived as gradually increasing rather than instantaneous, which can affect realism in scenarios involving rigid constraints.

However, this trade-off is necessary to ensure stable behaviour in the presence of communication delay. Without rate limiting and damping, the system exhibited oscillatory and unstable behaviour during contact, particularly under high stiffness settings, as demonstrated in Sections 5.3.2 and 5.4.2. The implemented approach therefore prioritises stability and controllability over maximum stiffness, which is appropriate for a low-cost prototype system.

These findings highlight a key design consideration in practical haptic systems: achieving stable interaction under real-world constraints often requires compromising on ideal force rendering performance.

6.5 Project Management and Development Process

Project management in this work was not followed as a rigid, pre-defined process, but evolved as technical challenges emerged. While an initial project plan was created to outline development stages, the Gantt chart included in Appendix B represents the final execution of the project. Development of the host software began prior to the official project start, with initial work on OpenGL-based rendering and interaction undertaken in late August, providing a foundation for subsequent system integration.

The original plan provided a useful structure, identifying phases such as hardware development, embedded control, communication, and host-side software. However, several aspects required more time than anticipated, particularly communication debugging, subsystem integration, and stability tuning, so the plan was used as a high-level guide rather than a strict schedule. This included a major refactoring of the host software architecture between January and March, improving modularity and separation of system components.

Significant deviations from the original plan occurred. Early integration difficulties with the selected motor drivers led to the adoption of SimpleFOC Mini drivers, prioritising reliable integration over calibrated torque control. Later, the failure of one actuator prevented full two-degree-of-freedom validation, requiring a shift to single-axis testing and system-level evaluation.

Risk management was primarily reactive, although key risks were identified early (Appendix C), including control instability, communication failure, and actuator limitations. As these arose, mitigation strategies such as torque limiting, slew-rate limiting, and watchdog safety mechanisms were introduced.

Time management was shaped by both technical challenges and practical constraints. A significant portion of development time was spent integrating subsystems, including the host software, embedded controller, and actuators, where achieving reliable end-to-end behaviour required iterative debugging across hardware and software boundaries, becoming a primary bottleneck in the development process. Progress was also temporarily constrained by hardware availability, particularly while awaiting replacement motor drivers, during which effort was deliberately redirected towards report writing, literature review, and test design.

The project required significant self-directed learning, documented in the CPD log (Appendix D), including real-time systems, multithreaded architectures, physics simulation, and haptic control. This

informed key design decisions such as snapshot-based communication and damping-based stabilisation.

Overall, although the project did not strictly follow its initial plan, effective project management was demonstrated through adapting to challenges, prioritising critical subsystems, and maintaining progress towards a functional system.

7 Conclusions and Future Work

7.1 Conclusions

This project presented the design, implementation, and evaluation of a low-cost, modular haptic rendering system integrating a two-degree-of-freedom input device with a custom real-time simulation and visualisation framework.

The system successfully achieved its primary objective of enabling bidirectional interaction between a user-operated device and a virtual environment, with real-time computation of interaction forces and feedback through actuator torque control. The architecture was designed to prioritise modularity and extensibility, allowing clear separation between hardware, communication, simulation, and rendering subsystems.

Experimental results demonstrated that the system is capable of sustaining high update rates of approximately 1 kHz, with negligible packet loss at higher transmission frequencies and low host-side processing latency. However, communication analysis revealed a consistent round-trip delay of approximately 15–16 ms, which dominates the closed-loop response and imposes a fundamental limitation on achievable haptic performance.

Despite this latency, stable and controllable haptic interaction was achieved through a combination of architectural and control strategies. The use of snapshot-based communication ensured that the system operated on the most recent available data, mitigating the effects of burst-like packet delivery. In addition, the application of damping and torque rate limiting was critical in maintaining stability, preventing oscillatory behaviour during contact, and enabling sustained interaction with virtual constraints.

Simulation and hardware validation confirmed that the system is capable of rendering physically

consistent contact and constraint forces. The virtual coupling and proxy-based rendering approach produced expected unilateral contact behaviour and supported constrained motion along virtual surfaces. Hardware testing further demonstrated that stable interaction could be maintained under real-time conditions, even with communication variability and partial actuation.

However, the results also highlight an important trade-off between stability and force fidelity. While the system remains stable under delayed feedback, the achievable virtual stiffness is limited, and contact forces exhibit a finite rise time rather than instantaneous response. This reduces the realism of high-speed or high-stiffness interactions, particularly for rigid contact scenarios.

Overall, the project demonstrates that stable and usable haptic interaction can be achieved using low-cost hardware and a modular software architecture, even in the presence of significant communication latency. The findings emphasise that in practical systems, communication delay is often the dominant constraint, and that appropriate architectural and control design is essential to achieving stable performance.

7.2 Future Work

Several avenues for future development have been identified to improve system performance, fidelity, and extensibility.

A key priority is the reduction of communication latency. The current system relies on the ST-Link virtual COM interface, which was selected for its simplicity and ease of integration during early development. While this approach enabled rapid prototyping, the communication results in Sections 5.1.2 and 5.1.3 indicate that the communication pathway introduces buffering and scheduling delays that dominate the closed-loop response.

Future work should therefore investigate lower-latency communication architectures between the host and embedded controller. In particular, direct UART communication via a dedicated USB–UART bridge may reduce buffering effects associated with the virtual COM interface, while alternative approaches such as native USB implementations (e.g., bulk transfer), CAN bus, or Ethernet-based communication could provide more deterministic timing behaviour.

A systematic comparison of these communication approaches would allow the contribution of each component in the communication stack to be isolated and could provide a pathway toward significantly reducing round-trip latency and improving achievable haptic stiffness.

On the control side, incorporating task-space dynamics using an operational-space formulation [13] would allow the manipulator's own inertia and dynamics to be compensated during haptic interaction, improving the transparency of force rendering and enabling more physically accurate simulation of robotic manipulator behaviour.

Hardware improvements represent another important area for future development. The incorporation of current sensing would enable more accurate torque control, allow quantitative validation of rendered forces, and support more advanced control strategies including impedance and operational-space control formulations. Mechanical transmission design could also be improved: while GT2 timing belts provided a low-cost solution for torque amplification, they introduced slip and compliance under high load, and future designs could explore capstan drives for reduced backlash and improved force transmission. The use of higher torque actuators or geared motor systems would further improve the ability to render stiff virtual environments, and restoration of full two-degree-of-freedom actuation would improve interaction realism and allow more complex manipulation and constraint-based tasks to be explored.

Further improvements to the haptic rendering model could address the force-direction inconsistencies observed during large penetration. In particular, constraint-based contact solvers could enforce continuous proxy motion subject to geometric constraints across timesteps, reducing the surface-normal discontinuities that arise when the proxy is re-projected from scratch at each update.

From a software perspective, the modular architecture provides a strong foundation for extension toward more advanced simulation environments. Integration with external platforms such as NVIDIA Isaac Sim or MATLAB/Simulink would enable more realistic robotic scenarios and facilitate comparison with established robotic control frameworks.

Finally, a more comprehensive evaluation framework incorporating quantitative force measurement, user studies, and task-based performance metrics would provide deeper insight into the effectiveness of the system as a training tool.

Together, these developments would enable the system to evolve from a low-cost prototype into a more capable and general-purpose haptic interface platform for robotic interaction and training applications.

References

[1] G. S. Giri, Y. Maddahi, and K. Zareinia, "An Application-Based Review of Haptics Technology," *Robotics*, vol. 10, no. 1, p. 29, Feb. 5, 2021, ISSN: 2218-6581. DOI: 10.3390/robotics10010029 Accessed: Apr. 16, 2026. [Online]. Available: https://www.mdpi.com/2218-6581/10/1/29 (cited on pp. 1, 4, 6).

[2] M. Bergholz, M. Ferle, and B. M. Weber, "The benefits of haptic feedback in robot assisted surgery and their moderators: A meta-analysis," *Scientific Reports*, vol. 13, no. 1, p. 19 215, Nov. 6, 2023, ISSN: 2045-2322. DOI: 10.1038/s41598-023-46641-8 Accessed: Apr. 16, 2026. [Online]. Available: https://www.nature.com/articles/s41598-023-46641-8 (cited on p. 1).

[3] D. C. Ruspini, K. Kolarov, and O. Khatib, "The haptic display of complex graphical environments," in *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques - SIGGRAPH '97*, Not Known: ACM Press, 1997, pp. 345–352, ISBN: 978-0-89791-896-1. DOI: 10.1145/258734.258878 Accessed: Apr. 16, 2026. [Online]. Available: http://portal.acm.org/citation.cfm?doid=258734.258878 (cited on pp. 1, 5).

[4] R. Adams and B. Hannaford, "Stable haptic interaction with virtual environments," *IEEE Transactions on Robotics and Automation*, vol. 15, no. 3, pp. 465–474, Jun. 1999, ISSN: 1042296X. DOI: 10.1109/70.768179 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/768179/ (cited on pp. 1, 5, 11).

[5] M. O. Martinez et al., "3-D printed haptic devices for educational applications," in *2016 IEEE Haptics Symposium (HAPTICS)*, Apr. 2016, pp. 126–133. DOI: 10.1109/HAPTICS.2016.7463166 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/7463166 (cited on pp. 2, 3, 6).

[6] M. O. Martinez et al., "Open source, modular, customizable, 3-D printed kinesthetic haptic devices," in *2017 IEEE World Haptics Conference (WHC)*, Jun. 2017, pp. 142–147. DOI: 10.1109/WHC.2017.7989891 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7989891 (cited on pp. 2, 3, 6).

[7] A. Lavatelli, M. Bordegoni, and F. Ferrise, "Design of an open-source low cost 2DOF haptic device," Sep. 2014. DOI: 10.1109/MESA.2014.6935574 (cited on pp. 2, 3, 6).

[8] "Geomagic touch haptic device," Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-haptic-device (cited on p. 4).

[9] “Geomagic touch X haptic device,” Accessed: Apr. 16, 2026. [Online]. Available: https://shop.gomeasure3d.com/products/geomagic-touch-x-haptic-device (cited on p. 4).

[10] “Sales inquiries,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.forcedimension.com/contact/sales (cited on p. 5).

[11] “Inverse3 X,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.haply.co/inverse3x (cited on p. 5).

[12] C. Zilles and J. Salisbury, “A constraint-based god-object method for haptic display,” in *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*, vol. 3, Pittsburgh, PA, USA: IEEE Comput. Soc. Press, 1995, pp. 146–151, ISBN: 978-0-8186-7108-1. DOI: 10.1109/IROS.1995.525876 Accessed: Apr. 16, 2026. [Online]. Available: http://ieeexplore.ieee.org/document/525876/ (cited on p. 5).

[13] O. Khatib, “A unified approach for motion and force control of robot manipulators: The operational space formulation,” *IEEE Journal on Robotics and Automation*, vol. 3, no. 1, pp. 43–53, Feb. 1987, ISSN: 2374-8710. DOI: 10.1109/JRA.1987.1087068 Accessed: Apr. 16, 2026. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/1087068 (cited on pp. 5, 35).

[14] J. E. Colgate and G. G. Schenkel, “Passivity of a class of sampled-data systems: Application to haptic interfaces,” *Journal of Robotic Systems*, vol. 14, no. 1, pp. 37–47, 1997, ISSN: 1097-4563. DOI: 10.1002/(SICI)1097-4563(199701)14:1<37::AID-ROB4>3.0.CO;2-V Accessed: Apr. 18, 2026. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/%28SICI%291097-4563%28199701%2914%3A1%3C37%3A%3AAID-ROB4%3E3.0.CO%3B2-V (cited on p. 5).

[15] T-Motor, *GB2208 gimbal type brushless motor*, 2026. [Online]. Available: https://store.tmotor.com/product/gb2208-gimbal-type.html (cited on p. 8).

[16] ams AG, *AS5048A/AS5048B magnetic rotary encoder (14-bit angular position sensor)*, v1-11, manual, ams AG, Jan. 2018. Accessed: Apr. 16, 2026. [Online]. Available: https://ams-osram.com/products/sensors/position-sensors/ams-as5048a-high-resolution-position-sensor (cited on p. 8).

[17] ams AG, *AS5600 12-bit programmable contactless potentiometer*, v1-06, manual, ams AG, Jun. 2018. Accessed: Apr. 16, 2026. [Online]. Available: https://ams-osram.com/products/sensor-solutions/position-sensors/ams-as5600-position-sensor (cited on p. 8).

[18] “NUCLEO-G431RB | Product - STMicroelectronics,” Accessed: Apr. 16, 2026. [Online]. Available: https://www.st.com/en/evaluation-tools/nucleo-g431rb.html (cited on p. 9).

[19] A. Skuric et al., “SimpleFOC: A field oriented control (FOC) library for controlling brushless direct current (BLDC) and stepper motors,” *Journal of Open Source Software*, vol. 7, no. 74, p. 4232, 2022. DOI: 10.21105/joss.04232 [Online]. Available: https://doi.org/10.21105/joss.04232 (cited on p. 9).

[20] P. Vas, *Sensorless Vector and Direct Torque Control*. Oxford University Press, May 14, 1998, ISBN: 978-0-19-856465-2. DOI: 10.1093/oso/9780198564652.001.0001 Accessed: Apr. 16, 2026. [Online]. Available: https://doi.org/10.1093/oso/9780198564652.001.0001 (cited on p. 9).

[21] C. Ericson, *Real-Time Collision Detection*, 0th ed. CRC Press, Dec. 22, 2004, ISBN: 978-0-08-047414-4. DOI: 10.1201/b14581 Accessed: Apr. 16, 2026. [Online]. Available: https://www.taylorfrancis.com/books/9780080474144 (cited on p. 10).

Appendices

A Project outline

Design and Development of a Low-Cost 2-DOF Haptic Training Interface for Robotic Manipulator Simulation

Tyler Ward

October 2025

1 Background and Motivation

Robotic manipulators are increasingly used in manufacturing, surgical, and remote handling applications. However, operator training often relies on visual feedback alone, limiting spatial understanding and increasing the risk of errors when transitioning to physical systems.

Haptic feedback provides tactile and force based sensations that improve the realism and safety of training by allowing users to feel resistance and contact forces. Commercial haptic devices are often expensive and closed source, which makes them unsuitable for flexible training or academic development.

This project aims to create a low cost modular 2 DOF haptic input device that can send user motion commands to a simulated robotic manipulator and receive realistic force feedback. The system will be designed to support future integration with real robotic hardware, providing an accessible and scalable training platform.

2 Aim

To design, prototype and evaluate a compact low cost two degree of freedom haptic interface that can communicate in real time with a robotic manipulator simulation and provide accurate position sensing and responsive force feedback to the user.

3 Objectives

1. **Literature and Technology Review**
   A review will be carried out to identify existing haptic devices and relevant control methods. This will guide the selection of motors, encoders, and actuation strategies suitable for a low cost modular interface.
2. **Physics Simulation Development**
   A physics based simulation environment will be designed to model manipulator dynamics, haptic interactions, and constraint behavior.
3. **Hardware Design**

1

Fig. 7. Original project outline as submitted at the start of the project

B Gantt Chart

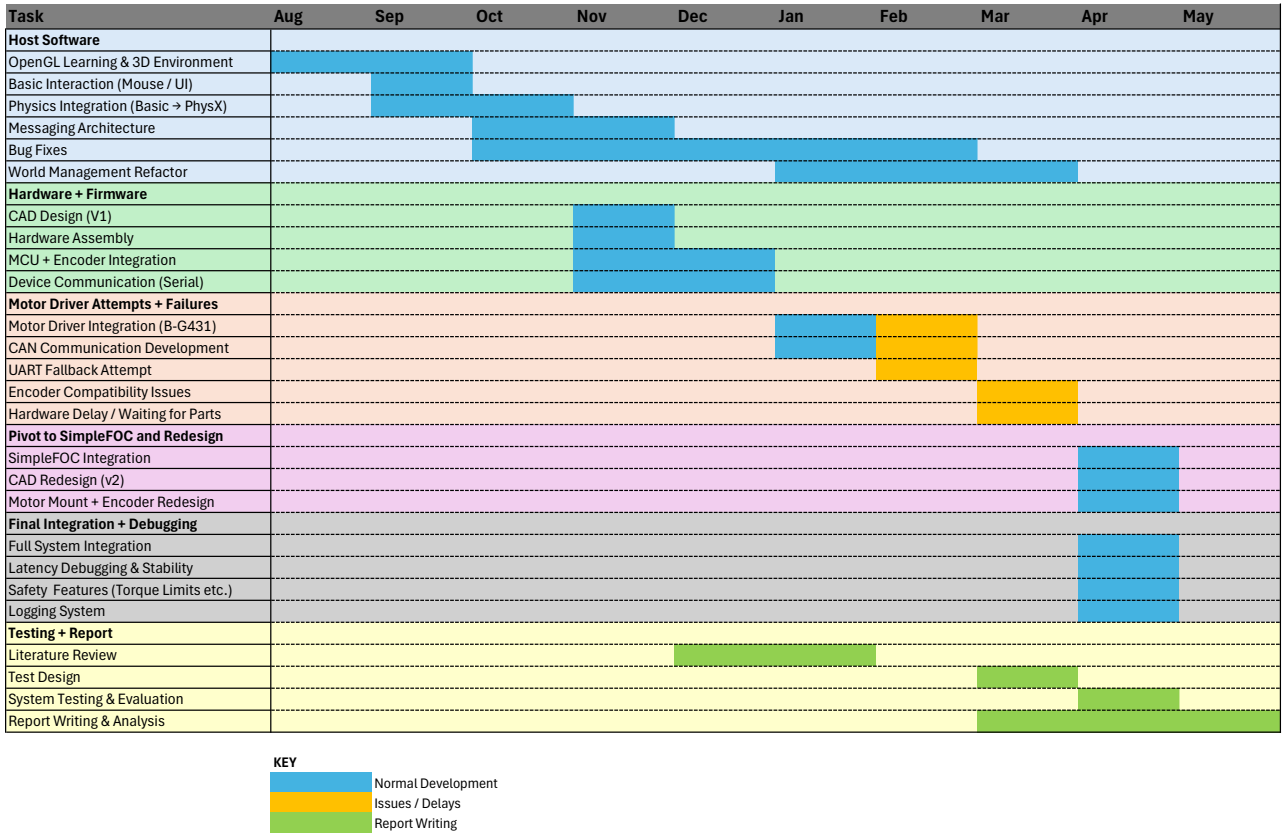


Fig. 8. Final Gantt chart showing the execution of the project over time. The chart reflects the iterative development process, including overlapping software and hardware work, as well as deviations from the original plan due to motor driver integration issues, hardware delays, and subsystem integration challenges.

C Risk assessment

MANCHESTER 1824
The University of Manchester

RISK REGISTER FOR 3rd YEAR PROJECT

Project Title:	**A Real-Time Haptic Rendering System Integrating Physics-Based Simulation and OpenGL Visualisation**		Submission Date:	1/05/2026	
Student Name:	11349510				

Project Risk	Severity			Potential			Score (Severity x Potential)	Mitigation Measures
	L	M	H	L	M	H	L=1, M=2, H=3	
Electrical faults during wiring/testing (short circuit, incorrect connections)	X				X		2	Use low-voltage systems; power off before modifying wiring; check connections before powering; use insulated connectors
Injury from moving mechanical parts (rotating joints, links)		X			X		4	Limit actuator torque; avoid contact during operation; test at low power; implement watchdog shutdown
Actuator overheating during prolonged operation		X		X			2	Monitor device temperature; limit runtime; allow cooling periods; avoid sustained high torque
Mechanical failure (belt slip, loose components)		X			X		4	Secure all components; inspect before testing; avoid excessive loading; operate within tested limits
Control instability causing unexpected motion or oscillations			X		X		6	Use conservative control gains; include damping; implement torque limits and rate limiting; test incrementally
Communication failure (loss of command stream)		X			X		4	Implement watchdog timeout to zero torque; validate communication before operation
Software bugs causing incorrect behaviour		X			X		4	Incremental testing; logging and debugging; isolate subsystems during development
Component failure (e.g. actuator failure during project)		X			X		4	Design for partial operation; maintain flexibility in testing; prioritise core functionality
Use of tools during assembly (cuts, slips)	X				X		2	Use appropriate tools; follow workshop safety procedures; maintain tidy workspace
Extended computer use (eye strain, fatigue)	X					X	3	Take regular breaks; maintain ergonomic posture; adjust screen setup

Fig. 9. Project Risk Assessment

D CPD Log

Continuing Professional Development Log

Name: *Tyler Ward*

Current and recent CPD activity:

CPD Activity Title	Description	Dates	CPD Hours
OpenGL and Real-Time Rendering	Self-taught OpenGL, shaders, and 3D rendering concepts using online resources.	Aug 2025 – Oct 2025	~20 hours
Haptic Rendering Methods	Read academic papers on virtual coupling and proxy-based haptics to understand force rendering approaches.	Aug 2025 – Jan 2026	~20hrs
BLDC Motor Control (SimpleFOC)	Learned to use SimpleFOC library for BLDC motor control and torque-mode operation.	Jan 2026 – Mar 2026	~10 hours
PhysX Simulation Integration	Learned how to integrate and use a physics engine (PhysX) for real-time simulation.	Jan 2026 – Feb 2026	~10hrs
Hardware Integration and Iterative Design	Gained practical experience designing and refining a mechanical system with motors and encoders.	Nov 2026 – Mar 2026	~20hrs
Working with a Large Codebase	Gained experience developing and managing a multi-component C++ codebase, including structuring modules, debugging across subsystems, and maintaining clarity in a multi-threaded architecture.	Aug 2026 – Apr 2026	~50 hrs
Technical Report Writing and Literature Review	Developed skills in structuring a technical report and conducting a focused literature review, including synthesising multiple sources, strengthening technical claims with references, and improving clarity and precision in written communication.	Dec 2025 – May 2026	~25 hrs

Planned and Future CPD activity:

CPD Activity Title	Description	Skills addressed	Dates
Technical Research and Literature Skills	Further develop ability to read and interpret technical research papers, including understanding methodologies, extracting key insights, and critically evaluating approaches.	Research skills, critical analysis, technical understanding	Summer 2026
Long-Term Project Planning	Improve ability to plan and manage longer-term technical projects, including setting realistic milestones, managing uncertainty, and adapting plans as challenges arise.	Project management, planning, problem-solving	Summer 2026

Fig. 10. Project Risk Assessment

E Mechanical Design Drawing

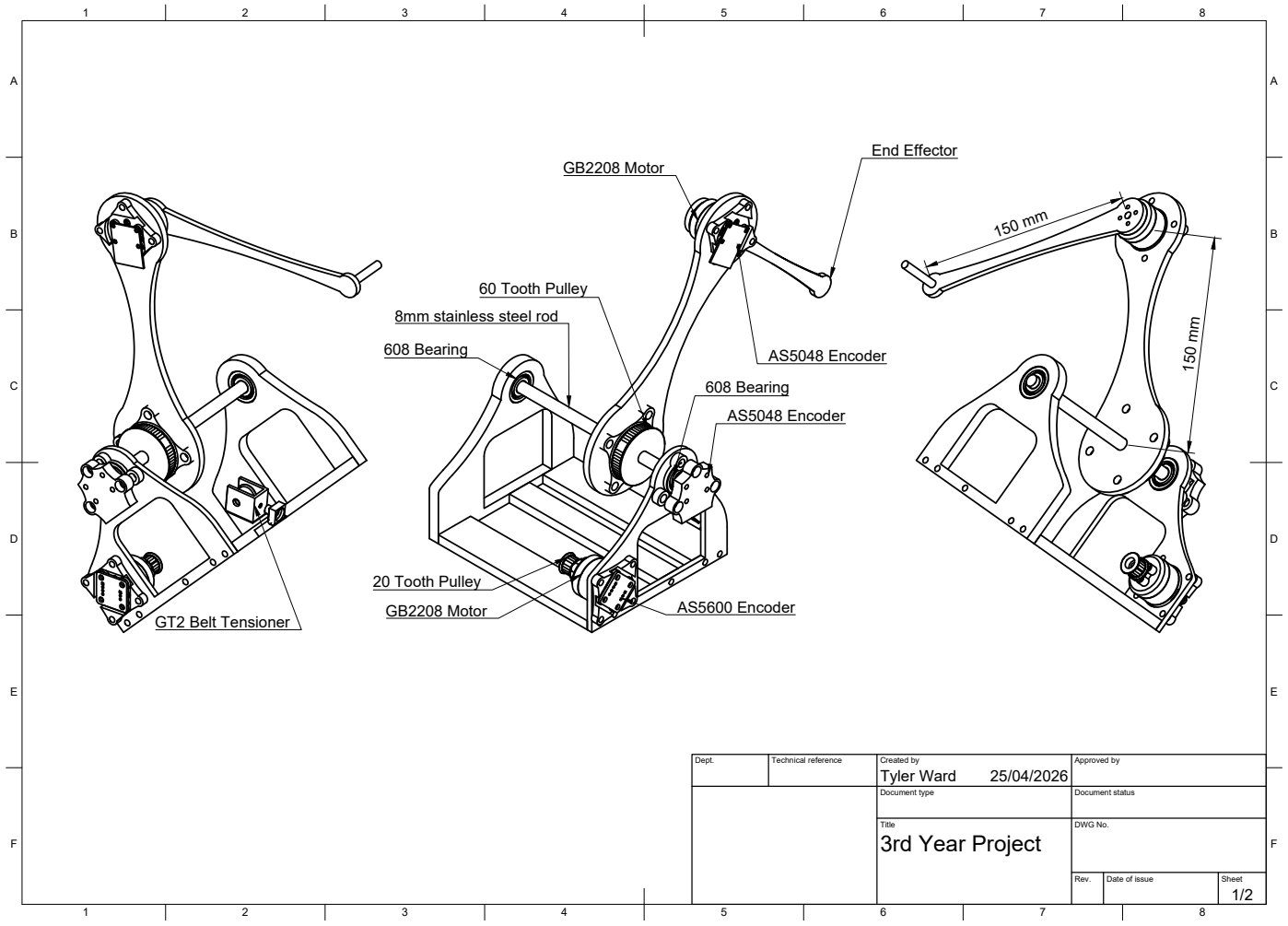


Fig. 11. CAD assembly of the 2-DOF haptic interface illustrating link geometry, actuator configuration, belt-driven base joint, and encoder placement. Link dimensions are annotated to define the kinematic model used for forward kinematics and Jacobian-based torque mapping.

F Code

Below is the README for the repo:

Real-Time Haptic Rendering System

Repository: https://github.com/TylerWardUOM/3rd-yr-project

Introduction

This project implements a real-time haptic rendering system that allows a user to physically feel virtual 3D objects through a custom-built 2-DOF planar robot arm. The system tracks the position of the device's end-effector and uses that as the position of a virtual "tool" inside a simulated 3D environment. When the tool touches a virtual object, a force is calculated and fed back to the physical device — giving the user the sensation of actually pushing against something.

The software is written entirely in C++17 and runs on Windows. It brings together four real-time concerns — rendering, physics simulation, haptic force computation, and hardware communication — each running on its own thread and communicating through a shared message-bus. The rendering thread provides a live OpenGL 3D view of the scene with an ImGui control panel. The physics thread runs NVIDIA PhysX at 240 Hz for rigid-body dynamics. The haptic thread runs at 1 kHz to compute contact forces using Signed Distance Functions (SDFs). The device thread also runs at 1 kHz, reading joint angles from the robot over serial and sending torque commands back.

System Overview

```
+-------------------------------------------------------------------+
|                          Application                              |
|                                                                   |
|  Main Thread      Sim Thread        Haptic Thread   Device Thread  |
```

```
|  -----------      ----------        ------------    -------------  |
|  GlSceneRenderer  WorldManager      HapticEngine    DeviceAdapter  |
|  + ImGui UI       PhysicsEnginePhysX   (1 kHz)      SerialLink     |
|  + Camera         (30 Hz / 240 Hz sub-step)          (1 kHz)       |
+-----------------------------------------------------------------+

Message Bus channels:
  world.snapshots    -->  Renderer, Physics
  haptics.tool_in    -->  HapticEngine
  haptics.wrenches   -->  PhysicsEngine
  device.wrench_cmd  -->  DeviceAdapter
  haptics.snapshots  -->  Renderer (overlay)
```

A demo video of early kinematics testing is available in the repository at: `docs/Media/Kinematics Demo 2025-09-20.mp4`

> Note: Still need to get a better more upto date video

Installation Instructions

These instructions assume a clean Windows PC with no prior project dependencies installed.

1. Install Prerequisites

| Tool | Where to get it | |—|—| | Visual Studio 2022 (with C++ Desktop workload) | https://visualstudio.microsoft.com/downloads/ | | CMake 3.20 or newer | https://cmake.org/download/ | | Git | https://git-scm.com/downloads | | vcpkg | https://github.com/microsoft/vcpkg |

Install vcpkg (package manager for C++ libraries):

```
git clone https://github.com/microsoft/vcpkg.git C:\Users\<you>\vcpkg
cd C:\Users\<you>\vcpkg
bootstrap-vcpkg.bat
```

Install PhysX via vcpkg:

```
vcpkg install physx:x64-windows
```

2. Clone the Repository

```
git clone --recurse-submodules https://github.com/TylerWardUOM/3rd-yr-project.git
cd 3rd-yr-project
```

If you already cloned without submodules, run:

```
git submodule update --init --recursive
```

This pulls in the vendored `imgui` and `glm` libraries automatically.

3. Update the vcpkg Path

Open `CMakeLists.txt` and change this line to match where your vcpkg is installed:

```
set(VCPKG_INSTALLED_DIR "C:/Users/<you>/vcpkg/installed/x64-windows")
```

4. Configure and Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

This produces two executables inside the build folder:

- `app.exe` — the main haptic rendering application
- `serial_tests.exe` — standalone serial communication test tool

5. Hardware (Optional)

To use the physical haptic device you will also need:

- A 2-DOF planar robot arm with quadrature encoders and BLDC motors, connected by USB serial
- The matching firmware flashed to the device's MCU (not included in this repo)
- COM port access (default: COM4, 460800 baud — configurable in `src/main.cpp`)

The application will run without the hardware connected; it will fall back to mouse-based tool control.

How to Run

Main Application

```
cd build\Release
app.exe
```

A window opens showing a 3D scene with a ground plane, a sphere, and a cube.

Controls:

| Input | Action | |—|—| | Right-click + drag | Look around (mouse-look) | | W / A / S / D | Fly camera | | Scroll wheel | Zoom | | ImGui panel (right side) | Select and move objects, change physics properties, adjust camera |

If the physical device is connected on COM4 at 460800 baud, a green sphere (device end-effector) and a blue sphere (haptic proxy) will appear in the scene as overlays.

Serial Communication Tests

The `serial_tests` executable lets you measure serial link performance without running the full simulation.

Packet rate / integrity test (checks packets received per second):

```
serial_tests rate COM4 460800 30 rate_results.csv
```

Round-trip time (RTT) test (measures latency):

```
serial_tests rtt COM4 460800 1000 2 rtt_results.csv
```

Arguments: `<port> <baud> <duration_sec or count> <interval_ms> <output_csv>`

Results are written to CSV for analysis.

Technical Details

Haptic Contact Detection — Signed Distance Functions (SDF)

Each virtual object's surface is represented by a Signed Distance Function φ(x). The value is positive outside the surface, zero on it, and negative inside. The gradient of φ points along the outward surface normal.

Plane SDF — for a plane with unit normal n and origin distance d:

```
 (x) = n · x − d
```

Sphere SDF — for a sphere with centre c and radius r:

```
 (x) = |x − c| − r
```

Proxy projection — when the tool penetrates a surface (φ < 0), a haptic proxy is snapped to the nearest point on the surface:

```
x_proxy = x_tool −  (x_tool) · n̂
```

This is the *god-object / proxy* method for haptic rendering (Zilles & Salisbury, 1995).

Virtual Coupling (Force Feedback)

A virtual spring-damper couples the physical device position x_d to the constrained proxy position x_p. This produces the force fed back to the device:

```
F_cpl = K(x_p − x_d) + D( _p −  _d)
```

Parameters used in this implementation:

| Parameter | Value | Description | |—|—|—| | K | 500 N/m | Spring stiffness | | M | 0.02 kg | Virtual mass (for critical damping calc) | | D | 0.7 × 2√(KM) ≈ 8.9 N·s/m | ~70% of critical damping, for stability | | F_max | 31 N | Hard force clamp — device safety limit |

The equal and opposite reaction force (−F) is applied as an impulse to the corresponding PhysX rigid body each physics step, so virtual objects can be pushed around by the device.

Forward Kinematics

The physical device is a 2-DOF planar robot arm with equal link lengths L1 = L2 = 0.15 m. Joint angles q1 and q2 (in radians) are read from the encoders. The end-effector position is:

```
x = L1·cos(q1) + L2·cos(q1 + q2)
y = L1·sin(q1) + L2·sin(q1 + q2)
```

The end-effector orientation quaternion is a rotation of (q1 + q2) about the Z axis.

Jacobian Torque Computation

To render a Cartesian force F = [Fx, Fy] at the end-effector, joint torques are computed via the Jacobian transpose ($\tau = J^T \cdot F$):

```
J = [ −L1·sin(q1) − L2·sin(q1+q2),   −L2·sin(q1+q2) ]
    [  L1·cos(q1) + L2·cos(q1+q2),    L2·cos(q1+q2)  ]

 1 = J11·Fx + J21·Fy
 2 = J12·Fx + J22·Fy
```

These torques are packed into a `TorqueCommandPacket` (with header `0xAA 0x55` and a checksum) and sent over serial to the MCU.

Threading Architecture

| Thread | Task | Rate | |—|—|—| | Main | OpenGL render loop, ImGui, camera | Display refresh | | Simulation | WorldManager step, PhysX step | ~30 Hz (1/240 s substeps) | | Haptics | SDF contact search, proxy update, force computation | 1 kHz | | Device | Serial read/parse, FK, torque send | 1 kHz |

All inter-thread communication uses lock-free message bus channels (`Channel<T>` for queues, `SnapshotChannel<T>` for latest-value reads).

Logging

On shutdown, the application writes two CSV files to the working directory:

- `device_timing.csv` — per-cycle timestamps (rx parse, tool publish, wrench consume, serial write) and signal values (q1, q2, fx, fy, τ1, τ2)
- `device_state_log.csv` — every parsed state packet from the device (sequence number, MCU timestamp, joint angles)

These are useful for diagnosing latency and communication issues.

Known Issues / Future Improvements

- **Windows-only:** `SerialLink` uses the Win32 `HANDLE` API. Porting to Linux/macOS would require replacing this with a POSIX serial implementation.
- **Hardcoded COM port:** The device port (`COM4`) and baud rate (`460800`) are hardcoded in `src/main.cpp`. These should be made configurable via a config file or command-line argument.
- **Hardcoded vcpkg path:** `CMakeLists.txt` has an absolute path to the PhysX install location. This should be replaced with a proper CMake toolchain file or `find_package` call.
- **No haptic loop stop flag:** `HapticEngine::run()` has no stop flag — the thread is detached rather than joined on shutdown. A clean shutdown flag should be added.
- **2D device only:** The current hardware is limited to a planar (XY) workspace. Extending to a 3-DOF or 6-DOF device would require updated kinematics and Jacobian.
- **No friction / texture rendering:** Only normal (penetration) forces are currently rendered. Surface friction, texture variation, and damping variation across surfaces are not yet implemented.
- **Mouse control limited:** Mouse-based tool control (fallback without the device) does not provide velocity data, so the damping term in the virtual coupling force is zero in that mode.
- **Actor rebuild is a full rebuild:** Any topology or physics property change causes all PhysX actors to be recreated from scratch. Incremental updates would be more efficient for larger scenes.